

Assignment 4

Author: Lan Stare Date: 3. 10. 2025

(a)

```
In [23]: import math

A = [[4, 12, -16],
      [12, 37, -43],
      [-16, -43, 98]] #example from question (b)

def cholensky_decomposition(matrix):
    m = len(matrix)
    L = [[0.0] * m for _ in range(m)] #first we build a zero m * m matrix where the zeros are floats
    L[0][0] = (math.sqrt(matrix[0][0])) #We then construct the element L_11 from the formula

    for i in range(m):
        for j in range(i + 1): #Note: we could also go from 1 to m + 1 and from 1 to i + 2, but in python doing it like this is easier and cleaner (starting from zero)
            if i == j:
                s = sum(L[i][k] ** 2 for k in range(j))
                L[i][j] = math.sqrt(A[i][i] - s) #we calculate the diagonal elements by formula
            else:
                s = sum(L[i][k] * L[j][k] for k in range(j))
                L[i][j] = (matrix[i][j] - s) / L[j][j] #we calculate the elements under the diagonal by formula

    return L

print(cholensky_decomposition(A))

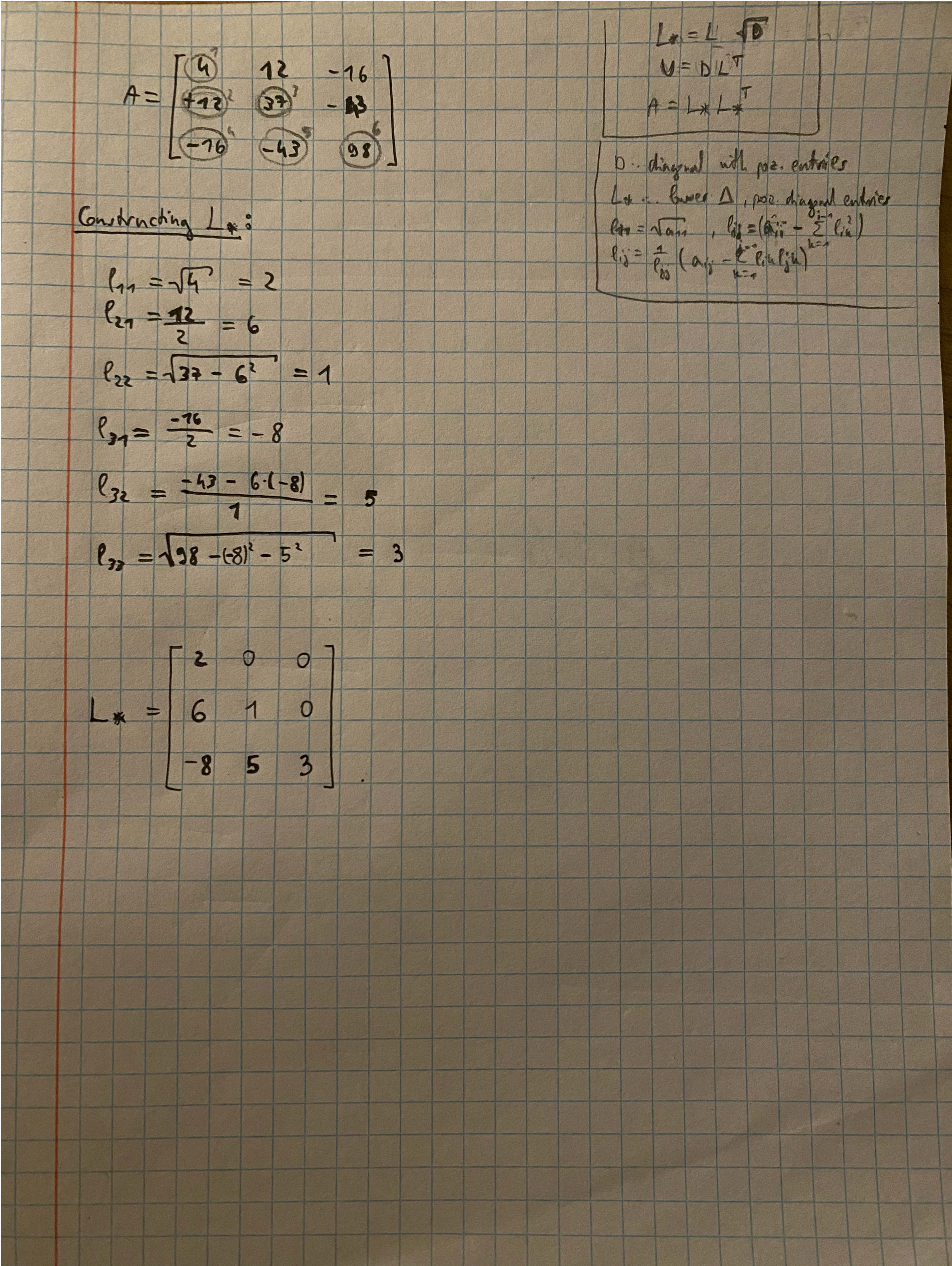
[[2.0, 0.0, 0.0], [6.0, 1.0, 0.0], [-8.0, 5.0, 3.0]]
```

(b)

```
In [5]: from IPython.display import Image, display

# Replace with your file path
image_path = "Cholensky.jpeg"

display(Image(filename=image_path))
```



Comparison: The matrices match! This is a good sign of both my hand precision and my algorithm working as they should, since the matrix L is unique.

(c)

```
In [1]: from scipy import linalg
import numpy as np
import time
import matplotlib.pyplot as plt

def build_example_matrix(i):
    m = 2 ** i
    main_diag = 2 * np.ones(m)
    off_diag = -1 * np.ones(m - 1) #The elements beside the diagonal are -1
    matrix = np.diag(main_diag) + np.diag(off_diag, k=1) + np.diag(off_diag, k=-1) # this is how the np combines the diagonal matrices

    return matrix

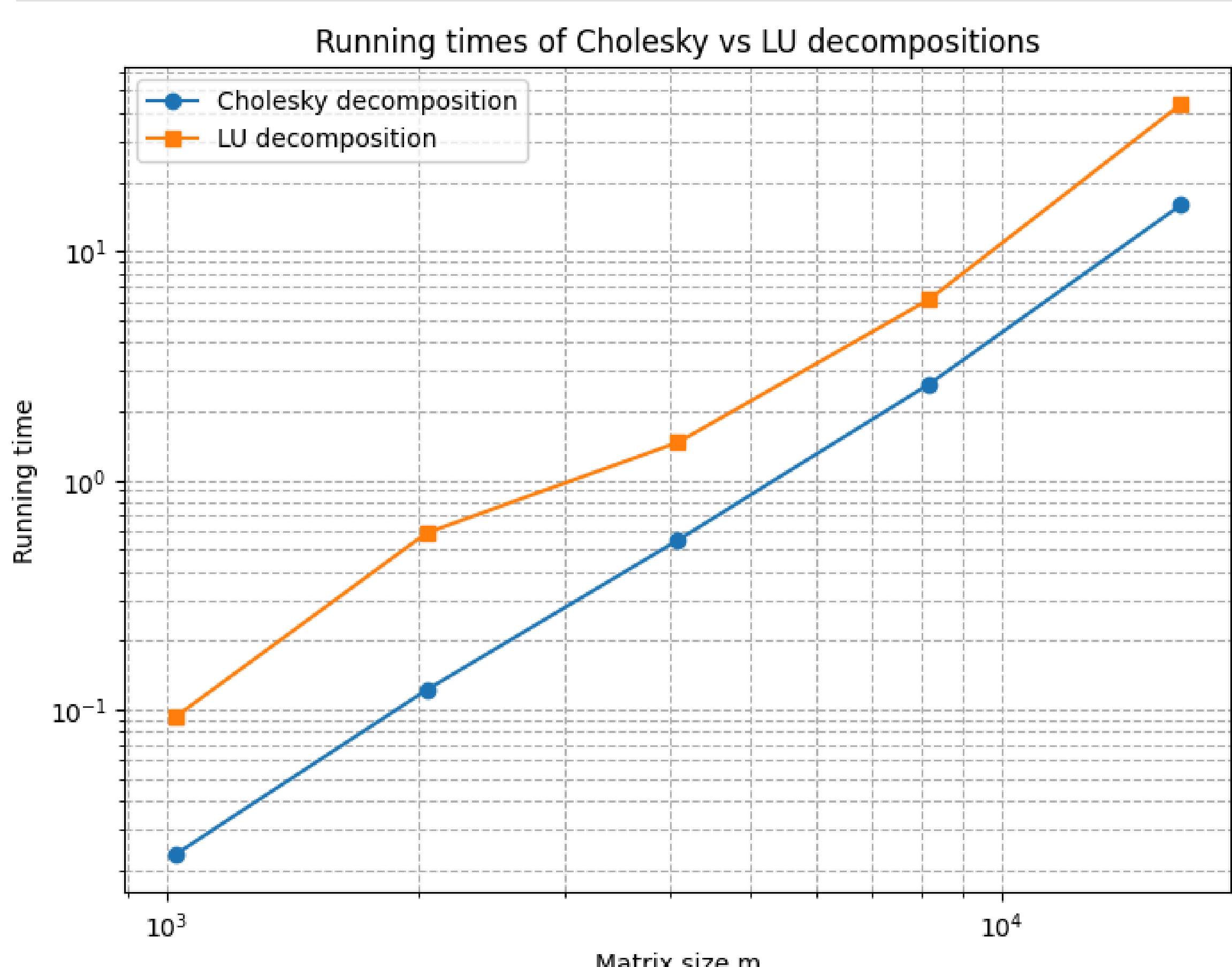
# Let's build the Cholesky and LU decompositions
def build_cholesky_and_LU(matrix):
    cholesky = linalg.cholesky(matrix)
    P, L, U = linalg.lu(matrix)
    return cholesky, (P, L, U)

i_values = range(10, 15)
m_values = [2 ** i for i in i_values]
times_cholesky = []
times_lu = []

for i, m in zip(i_values, m_values):
    A = build_example_matrix(i)
    # Measure Cholesky time
    start = time.time()
    linalg.cholesky(A) # this builds a Cholesky decomposition
    end = time.time()
    times_cholesky.append(end - start)
    # Measure LU time
    start = time.time()
    linalg.lu(A) # this builds an LU decomposition
    end = time.time()
    times_lu.append(end - start)

# now we can start plotting
plt.figure(figsize=(8, 6))
plt.loglog(m_values, times_cholesky, 'o-', label='Cholesky decomposition')
plt.loglog(m_values, times_lu, 's-', label='LU decomposition')
plt.xlabel('Matrix size m')
plt.ylabel('Running time')
plt.title('Running times of Cholesky vs LU decompositions')
plt.legend()
plt.grid(True, which="both", ls="--")
plt.show()

# this is so we can see the results
print("\nResults:")
for m, tc, tl in zip(m_values, times_cholesky, times_lu):
    print(f"m = {m:6d}: Cholesky = {tc:.6f}s, LU = {tl:.6f}s")
```



Results:

m	Cholesky	LU
1024	0.023300s	0.092650s
2048	0.121723s	0.589784s
4096	0.548650s	1.462444s
8192	2.629852s	6.160318s
16384	15.867758s	43.570057s

From the lectures we already know that the number of flops for the LU decomposition of m*m matrix is 2/3 m^3 and for the Cholesky decomposition about 1/3 m^3 flops. In the log-log plot of running times versus m, both curves should appear as roughly straight lines with a slope close to 3, reflecting the O(m^3) growth rate. The Cholesky curve is below the LU curve because of the smaller constant factor in its complexity (about 1/2 of LU's cost).